

Scalability & Future Plan

Date	15 July 2026
Team ID	XXXXXX
Project Name	FloodGuard - AI/ML Flood Prediction System
Maximum Marks	1 Mark

Current System Limitations

S.No	Limitation	Impact	Priority to Address
1	SQLite on ephemeral disk (Render free tier)	Prediction history is lost on every redeploy	High
2	Small, imbalanced training dataset (115 rows, 16 flood cases)	Model may not generalize well to regions/conditions outside the training data	High
3	No user authentication	All predictions are logged under a single guest account	Medium
4	Free-tier cold starts	First request after 15 min idle takes ~50-60 seconds	Medium
5	Manual weather-data entry	Users must know current readings; no live weather-API integration	Low

Scalability Plan

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Single free-tier instance, WEB_CONCURRENCY=1	Upgrade to a paid Render plan with multiple gunicorn workers
2	Data Storage	SQLite file on ephemeral disk	Migrate to Render's managed PostgreSQL for persistent history
3	Performance	Cold start after idle on free tier	Move to an always-on paid instance for production use
4	Security	No authentication; guest user only	Add proper user accounts and role-based access using the existing User table

Future Roadmap

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Migrate to PostgreSQL for persistent prediction history	1-2 weeks	History survives redeloys; enables long-term trend analysis
Phase 3	User authentication & role-based dashboards (admin vs viewer)	2-3 weeks	Personalized history, restricted access for sensitive regions
Phase 4	Live weather-API integration (auto-fill inputs by location)	3-4 weeks	Removes manual data entry, improves adoption by non-technical users